
Advanced NLP – Compression, Prompting, Instruction Tuning

Outline

- Model Compression
 - Reducing model size
- Fine-Tuning
 - Transfer Learning
- Instruction Tuning
- Retrieval Augmented Generation
 - LangChain?

What is Model Compression?

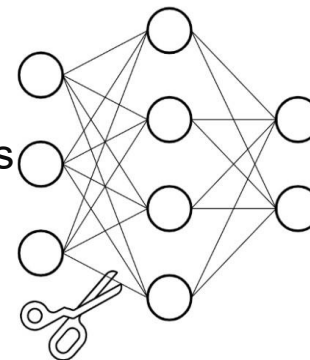
- The main idea is to **simplify** the model without **diminishing** accuracy. A simplified model means reduced in size and/or latency from the original. Both types of reduction are desirable.
 - **Size** reduction can be achieved by reducing the model parameters and thus using less RAM.
 - **Latency** reduction can be achieved by decreasing the time it takes for the model to make a prediction, and thus lowering energy consumption at runtime (and carbon footprint).

Compression Techniques (Algos)

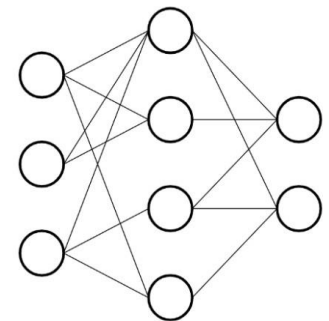
1. Pruning
2. Quantization
3. Knowledge distillation
4. Low-rank approximation and sparsity

Pruning

- The main idea is to **remove** features with nearly the same information.
- Pruning involves removing connections between neurons, channels, or filters from a trained network. To prune a connection, we set a weight in the matrix to zero. To prune a neuron, we set all values of a column to zero.
- **2 types of pruning:**
 - Unstructured removes connections or neurons
 - Structured removes filters or channels

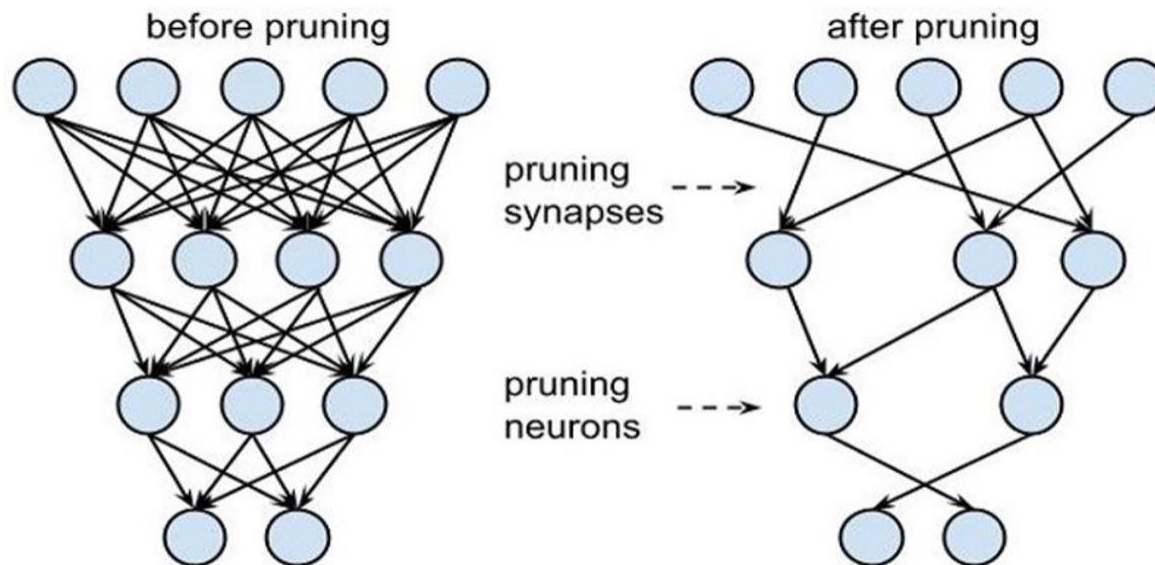


Before pruning



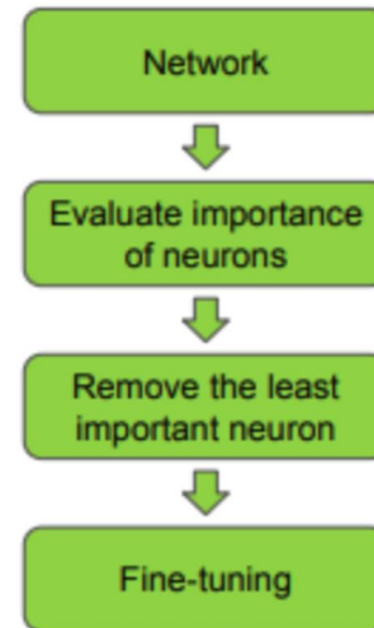
After pruning

Pruning



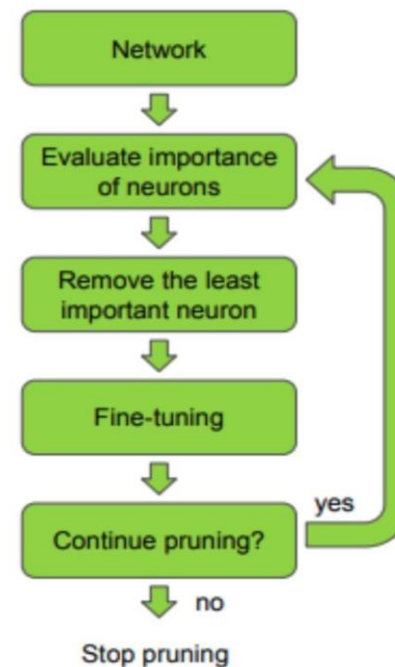
Types of Pruning

- **One-Shot Pruning**
- The network connections in this type of pruning are pruned only once.
- Steps:
 - Randomly initialize a neural network.
 - Train the network for certain iterations to find optimal weights.
 - Prune $p\%$ of weights from each layer in the model



Iterative pruning

- The network pruning is done partially through multiple iterations.
- Steps:
 - Randomly initialize a neural network.
 - Repeat for n rounds:
 - Train the network for certain iterations.
 - Prune $p(1/n)\%$ of weights that survived previous pruning



Lottery ticket hypothesis

- *A randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.*
- Lottery ticket hypothesis aims to find a subnetwork which has following properties:
 - Is there a subnetwork with better results
 - Shorter training time
 - Notably fewer parameters

Subnetwork with lottery ticket hypothesis

- Consider a dense feed-forward neural network $f(x; \theta)$ with initial parameters $\theta = \theta_0 \sim D_\theta$
- Where θ_0 is the chosen initialization parameters from the parameter space D_θ
- f reaches a minimum validation loss l at iteration j with test accuracy a
- In addition, consider training another network $f'(x; m \odot \theta)$ with a mask $m \in \{0, 1\}^{|\theta|}$ on its parameters such that initialization parameters are now $m \odot \theta_0$.
- On the same training set (with m fixed), f' reaches minimum validation loss l' at iteration j' with test accuracy a'
- $\exists m$ (mask) for which
 - $j' \leq j$ (*comparable training time*)
 - $a' \geq a$ (*comparable accuracy*)
 - $\|m\|_0 \ll |\theta|$ (*fewer parameters*)

Steps:

- **One-shot pruning:**

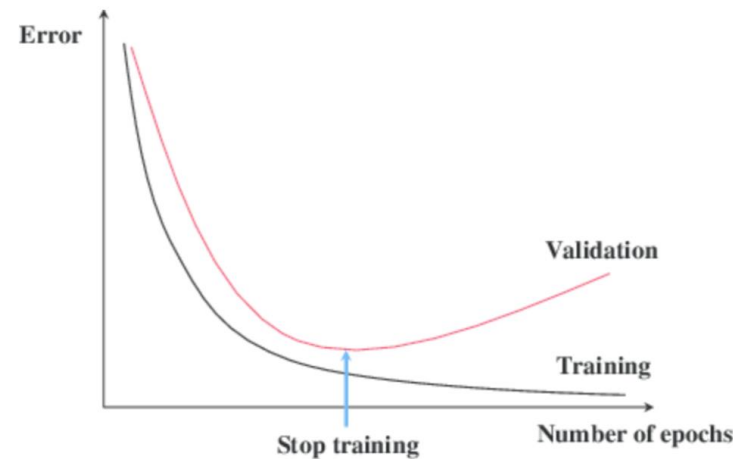
1. Randomly initialize a neural network $f(x; \theta_0)$, with initial parameters θ_0
2. Train the network for j iterations, arriving at parameters θ_j
3. Prune $p\%$ of the parameters in θ_j , creating a mask m
4. Reset the remaining parameters to their value in θ_0 , creating the winning ticket $f(x; m \odot \theta_0)$.

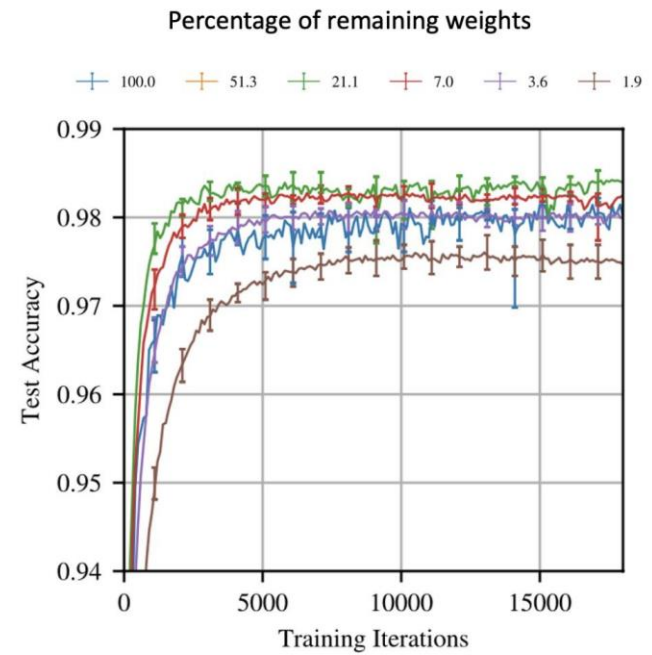
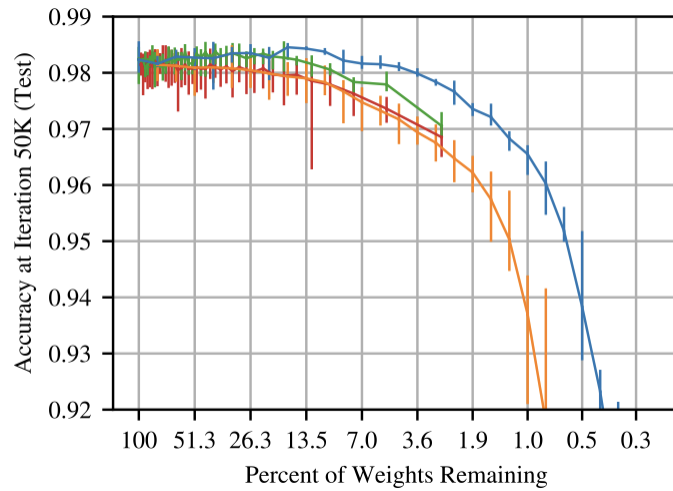
- **Iterative pruning Steps:**

1. Randomly initialize a neural network $f(x; \theta_0)$, with initial parameters θ_0
2. Train the network for j iterations, arriving at parameters θ_j
3. Prune $p1/n\%$ of the parameters in θ_j , creating a mask m
4. Reset the remaining parameters to their value in θ_0 , creating network $f(x; m \odot \theta_0)$
5. Repeat n times from 2
6. Final network is a winning ticket $f(x; m \odot \theta_0)$

Early Stopping

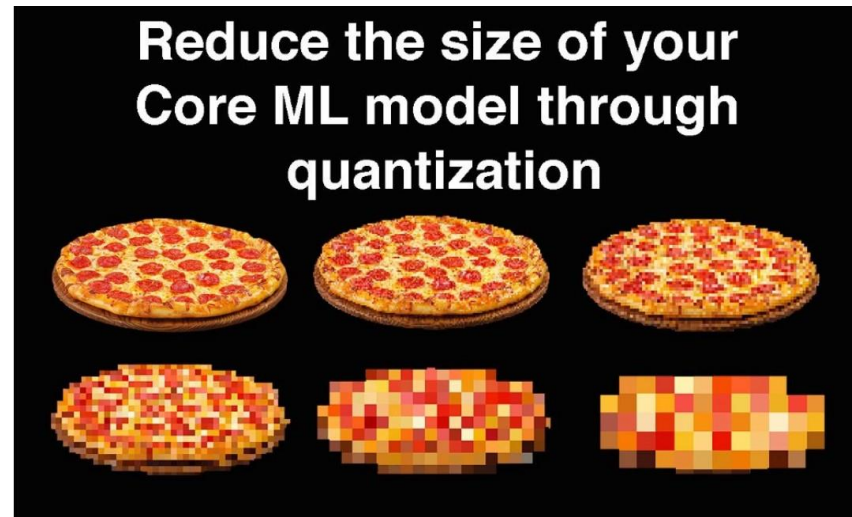
- To prevent the model from overfitting, the authors use early stopping as the convergence criteria. The iteration for early stopping is decided on the basis of validation loss.



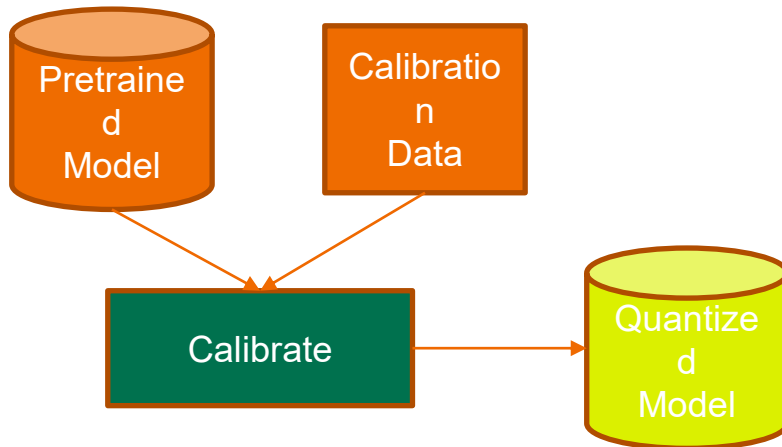


Quantization

- Map values from a large set to values in a smaller set
 - without losing too much information in the process.
 - So by reducing the number of pixels, the image below should still be clear.



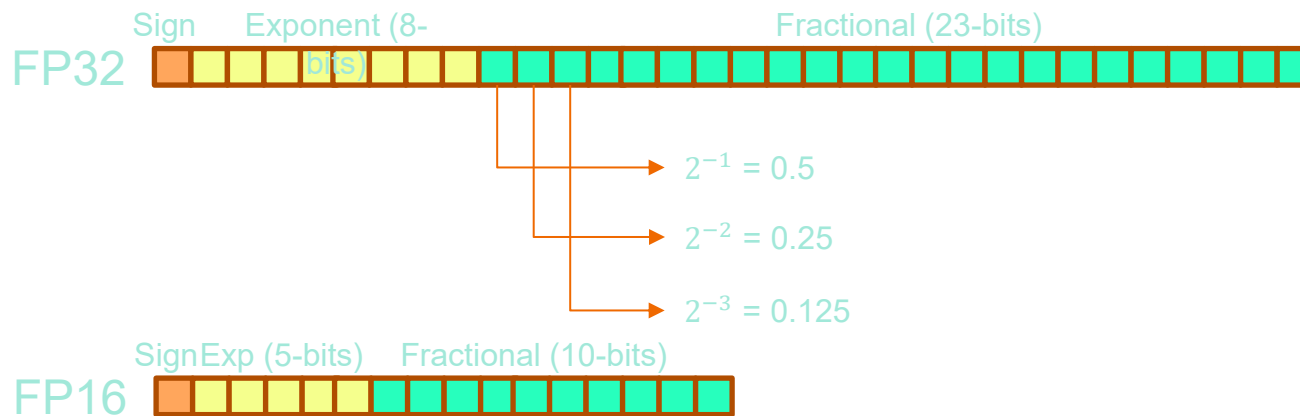
PTQ: Post Training Quantization



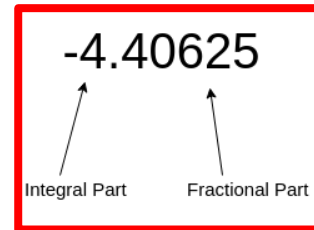
- GPTQ
- AdaQuant
- AdaRound
- BitSplit
- ZeroQuant
- SmoothQuant



Basic sensitivity study by Truncation



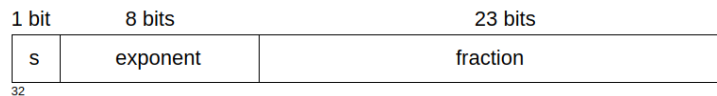
FP Representation



$0.40625 \times 2 = 0.8125 \gg 0$
 $0.8125 \times 2 = 1.625 \gg 1$
 $0.625 \times 2 = 1.25 \gg 1$
 $0.25 \times 2 = 0.5 \gg 0$
 $0.5 \times 2 = 1.0 \gg 1$

$0.40625_{10} = 0.01101_2$

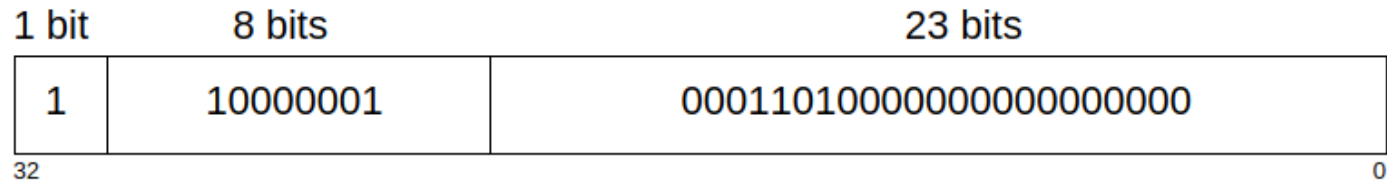
$$(-1)^s \times F \times 2^E$$



$$4_{10} = 100_2$$

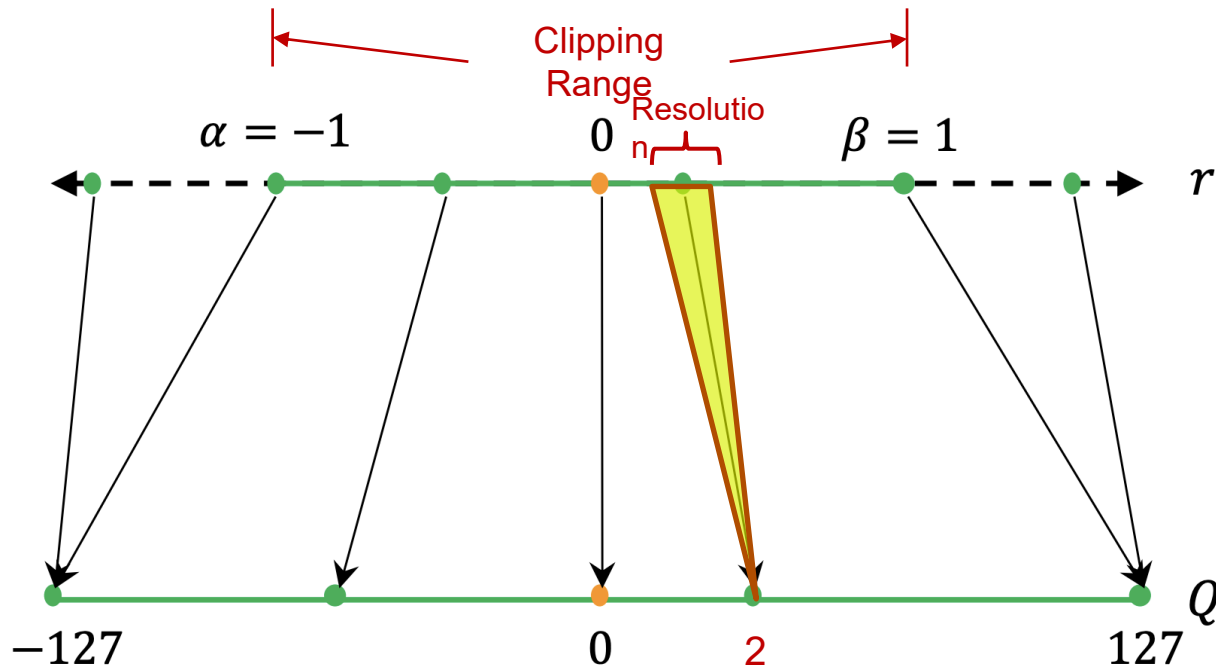
$$\text{Bias} = 2^{k-1} - 1$$

$$\text{exponent} = 2 + 2^{8-1} - 1 = 2 + 127 = 129 = 10000001_2$$





Uniform Quantization – INT8



$$S = \frac{\max(|\alpha|, |\beta|)}{2^{b-1} - 1}$$

$$Q(r) = \text{Int}\left(\frac{r}{S}\right)$$

$$\bar{r} = S(q)$$

INT4 in llama.cpp (Q4_0)

```
float amax = 0.0f; // absolute max

for (int l = 0; l < QK4_0; l++) {
    const float v = x[i*QK4_0 + l];
    amax = MAX(amax, fabsf(v));
}

const float d = amax / ((1 << 3) - 1);
const float id = d ? 1.0f/d : 0.0f;

y[i].d = d;

for (int l = 0; l < QK4_0; l++) {
    const float v0 = x[i*QK4_0 + l + 0]*id;
    const float v1 = x[i*QK4_0 + l + 1]*id;

    const uint8_t vi0 = (int8_t)roundf(v0) + 8;
    const uint8_t vi1 = (int8_t)roundf(v1) + 8;

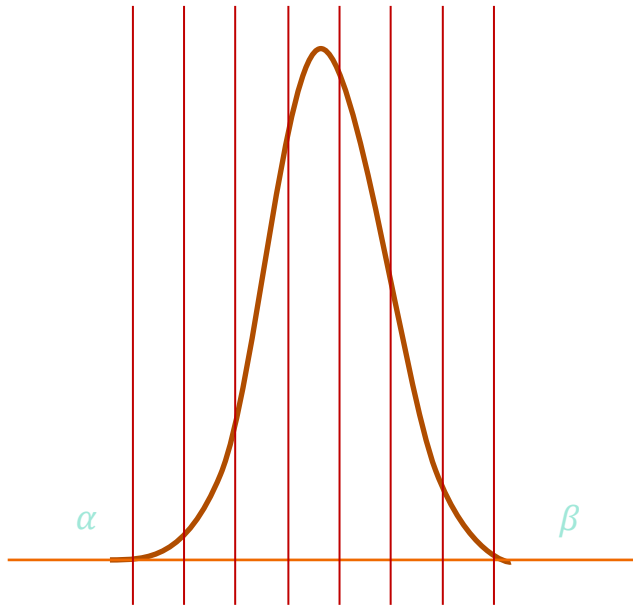
    assert(vi0 < 16);
    assert(vi1 < 16);

    pp[l/2] = vi0 | (vi1 << 4);
}
```

$$S = \frac{\max(|\alpha|, |\beta|)}{2^{b-1} - 1}$$

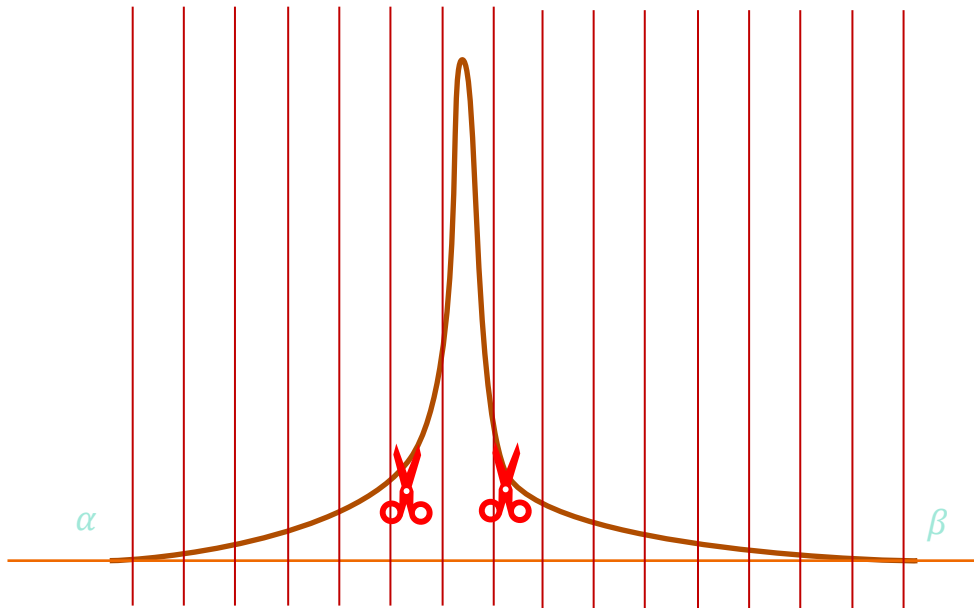
$$Q(r) = \text{Int}\left(\frac{r}{S}\right)$$

Distribution of weights matters



$$S = \frac{\max(|\alpha|, |\beta|)}{2^{b-1} - 1}$$

The Outlier Problem



1. Clip to create better α and β ?

2. Reduce granularity to per-tensor
than whole model ?

3. Reduce granularity even further to
per-row/col of the tensor ?

4. Use non-uniform quantization

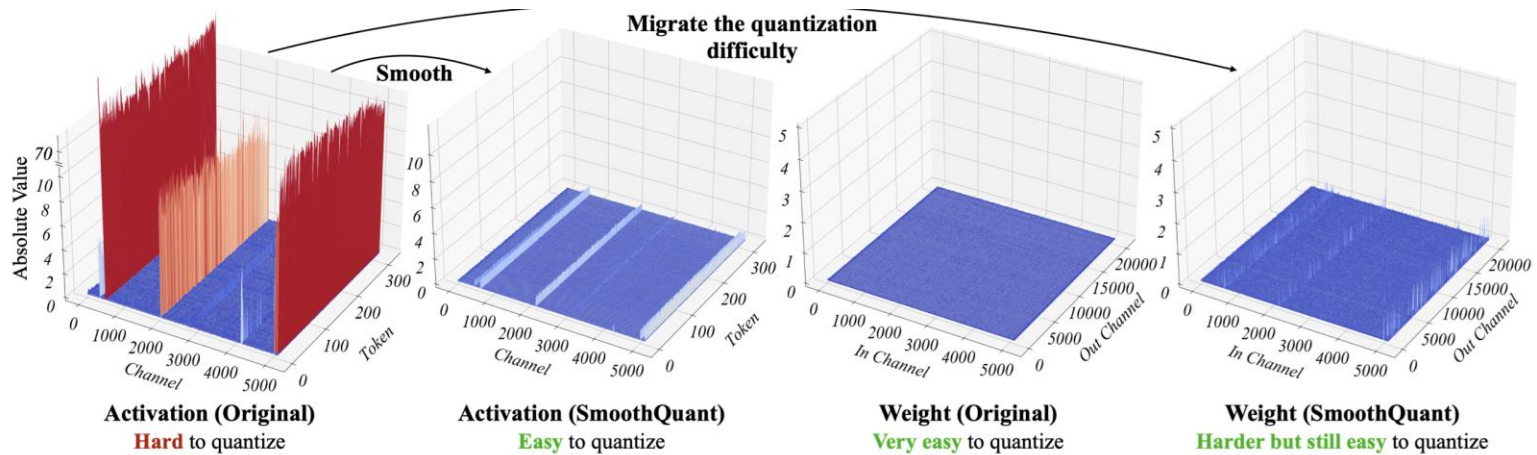
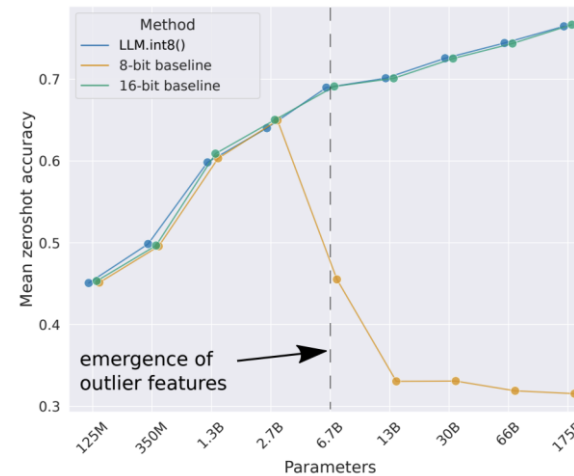
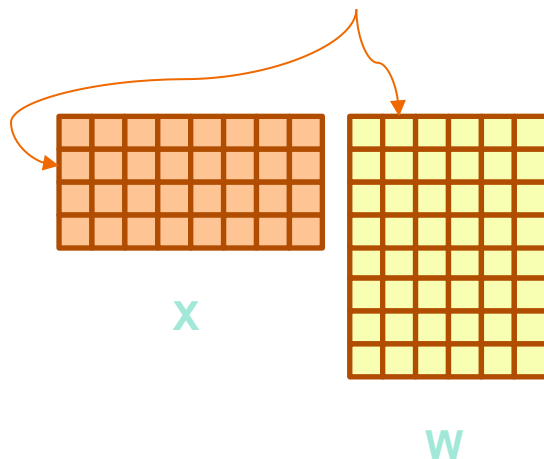


Figure 4: Magnitude of the input activations and weights of a linear layer in OPT-13B before and after SmoothQuant. Observations: (1) there are a few channels in the original activation map whose magnitudes are very large (greater than 70); (2) the variance in one activation channel is small; (3) the original weight distribution is flat and uniform. SmoothQuant migrates the outlier channels from activation to weight. In the end, the outliers in the activation are greatly smoothed while the weight is still pretty smooth and flat.

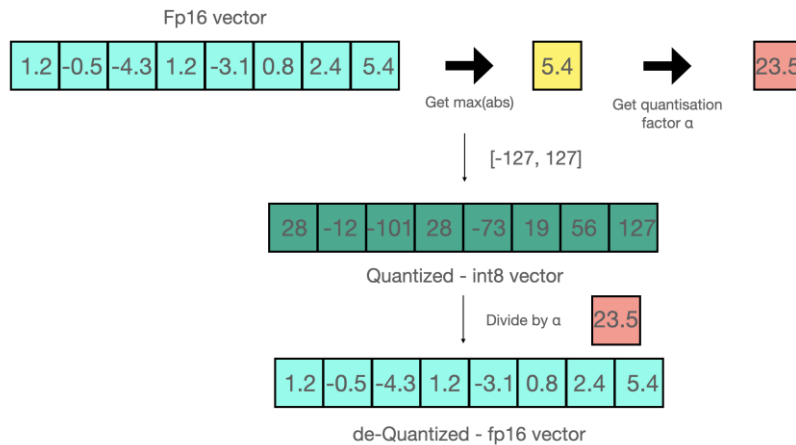
LLM.int8 – vector quantization limits

We quantize individual vectors



“Outlier features are probabilistic in smaller models, and systematic in larger”

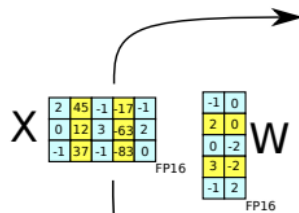
Traditional



$$\mathbf{X}_{i8} = \left\lceil \frac{127 \cdot \mathbf{X}_{f16}}{\max_{ij}(|\mathbf{X}_{f16_{ij}}|)} \right\rceil = \left\lceil \frac{127}{\|\mathbf{X}_{f16}\|_{\infty}} \mathbf{X}_{f16} \right\rceil = \left\lceil s_{x_{f16}} \mathbf{X}_{f16} \right\rceil,$$

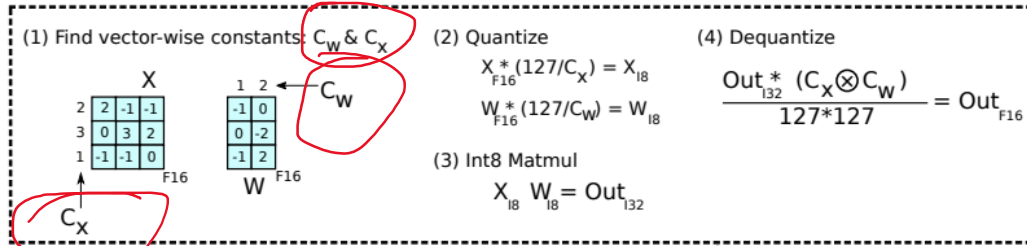
LLM.int8 – decompose outliers

LLM.int8()

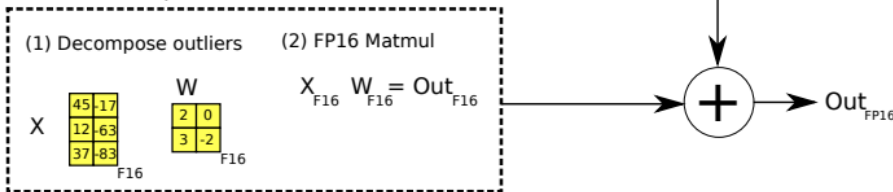


Regular values
Outliers

8-bit Vector-wise Quantization



16-bit Decomposition



Emergent Features

1. Emergence is not sudden but gradual and grows according to an exponential function related to perplexity and not model size.
2. Outlier features grow very quickly once their phase shift occurs.
3. The number of outliers features is strictly proportional to perplexity.

"Emergence is a gradual change in a property that suddenly undergoes a phase shift and then changes the quality of its substrate."

Emergent Features in Transformers

Substrate: Transformer

Property: Very large features in particular hidden dimensions across the transformer

Gradual change: Decreasing perplexity, more and larger outlier features

Phase shift: Outlier features suddenly become available in all transformer layers and coordinate through a few hidden dimensions.

Change of quality: Highly sparse, almost discrete attention; very dense FFN layers; “dual attention”; long-range attention (?); stable training through increased numerical stability.

$$\mathbf{C}_{f16} \approx \frac{1}{\mathbf{c}_{x_{f16}} \otimes \mathbf{c}_{w_{f16}}} \mathbf{C}_{i32} = S \cdot \mathbf{C}_{i32} = \mathbf{S} \cdot \mathbf{A}_{i8} \mathbf{B}_{i8} = \mathbf{S} \cdot Q(\mathbf{A}_{f16}) Q(\mathbf{B}_{f16}),$$

$$\mathbf{C}_{f16} \approx \sum_{h \in O} \mathbf{x}_{f16}^h \mathbf{w}_{f16}^h + \mathbf{S}_{f16} \cdot \sum_{h \notin O} \mathbf{x}_{i8}^h \mathbf{w}_{i8}^h$$

Smooth Quant

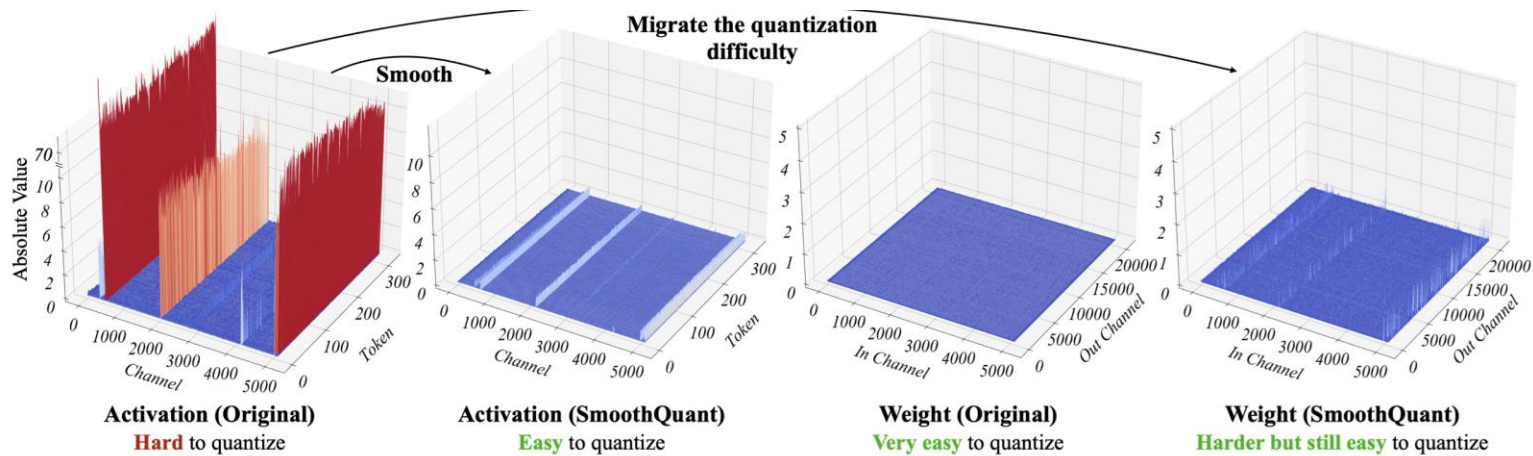
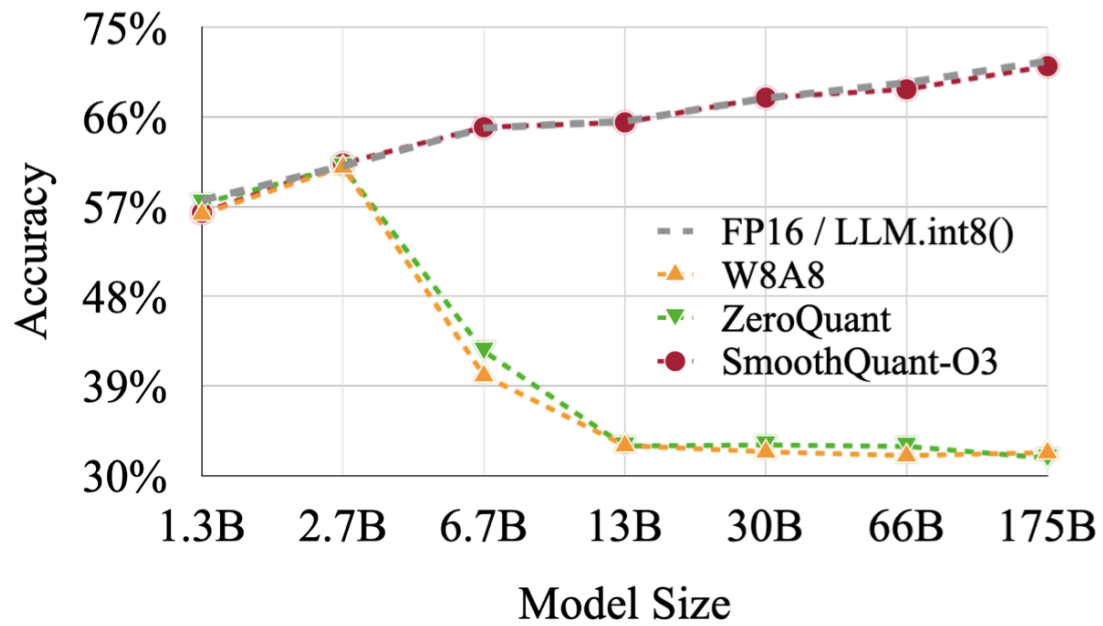
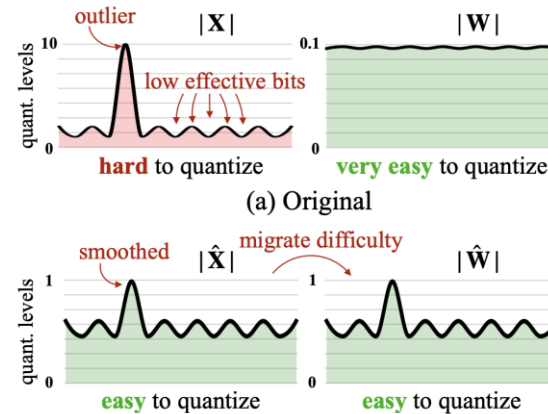
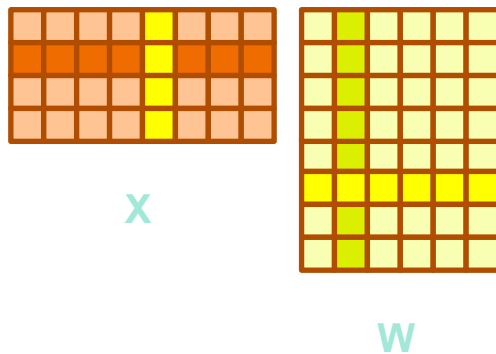


Figure 4: Magnitude of the input activations and weights of a linear layer in OPT-13B before and after SmoothQuant. Observations: (1) there are a few channels in the original activation map whose magnitudes are very large (greater than 70); (2) the variance in one activation channel is small; (3) the original weight distribution is flat and uniform. SmoothQuant migrates the outlier channels from activation to weight. In the end, the outliers in the activation are greatly smoothed while the weight is still pretty smooth and flat.



SmoothQuant – a different approach



$$s_j = \max(|\mathbf{X}_j|)^\alpha / \max(|\mathbf{W}_j|)^{1-\alpha}$$

$$Y = XY \cong (X \text{diag}(s)^{-1})(\text{diag}(s)W)$$

Precision

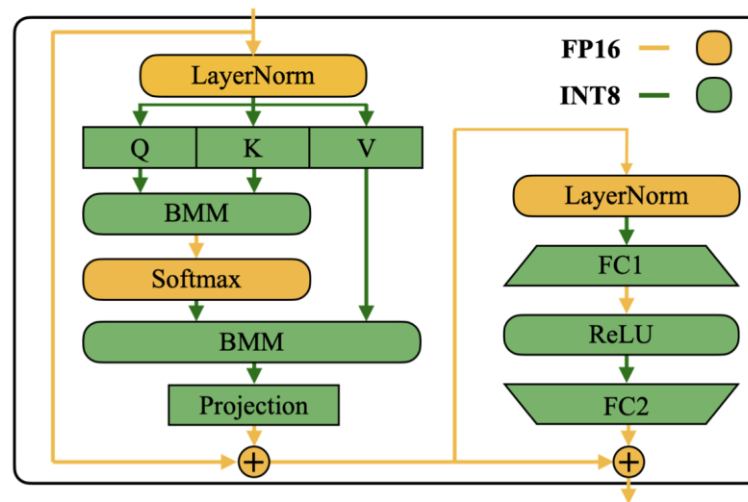
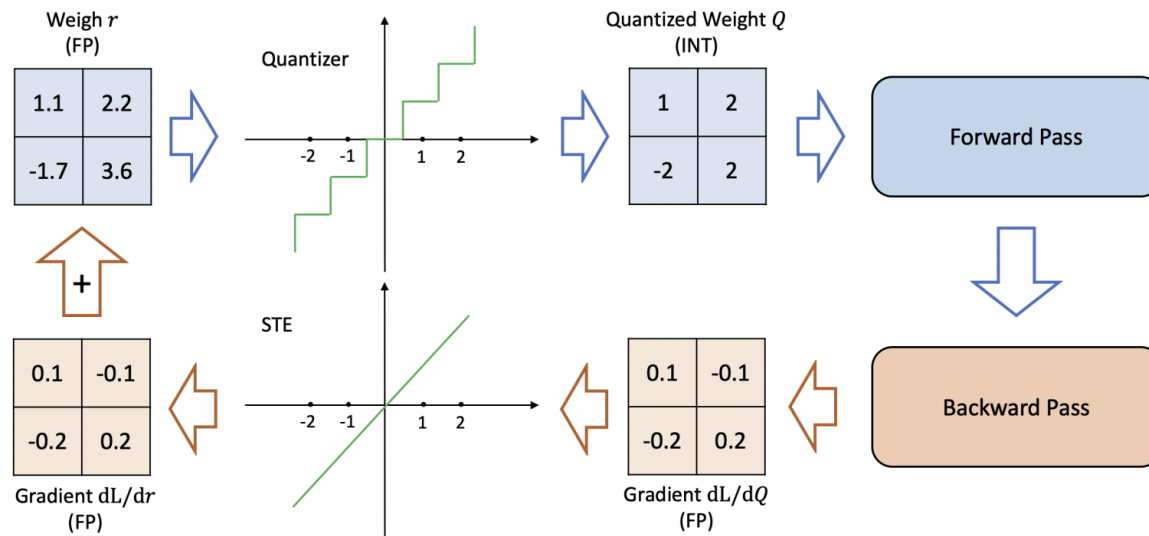


Figure 6: SmoothQuant's precision mapping for a Transformer block. All compute-intensive operators like linear layers and batched matmul (BMMs) use INT8 arithmetic.

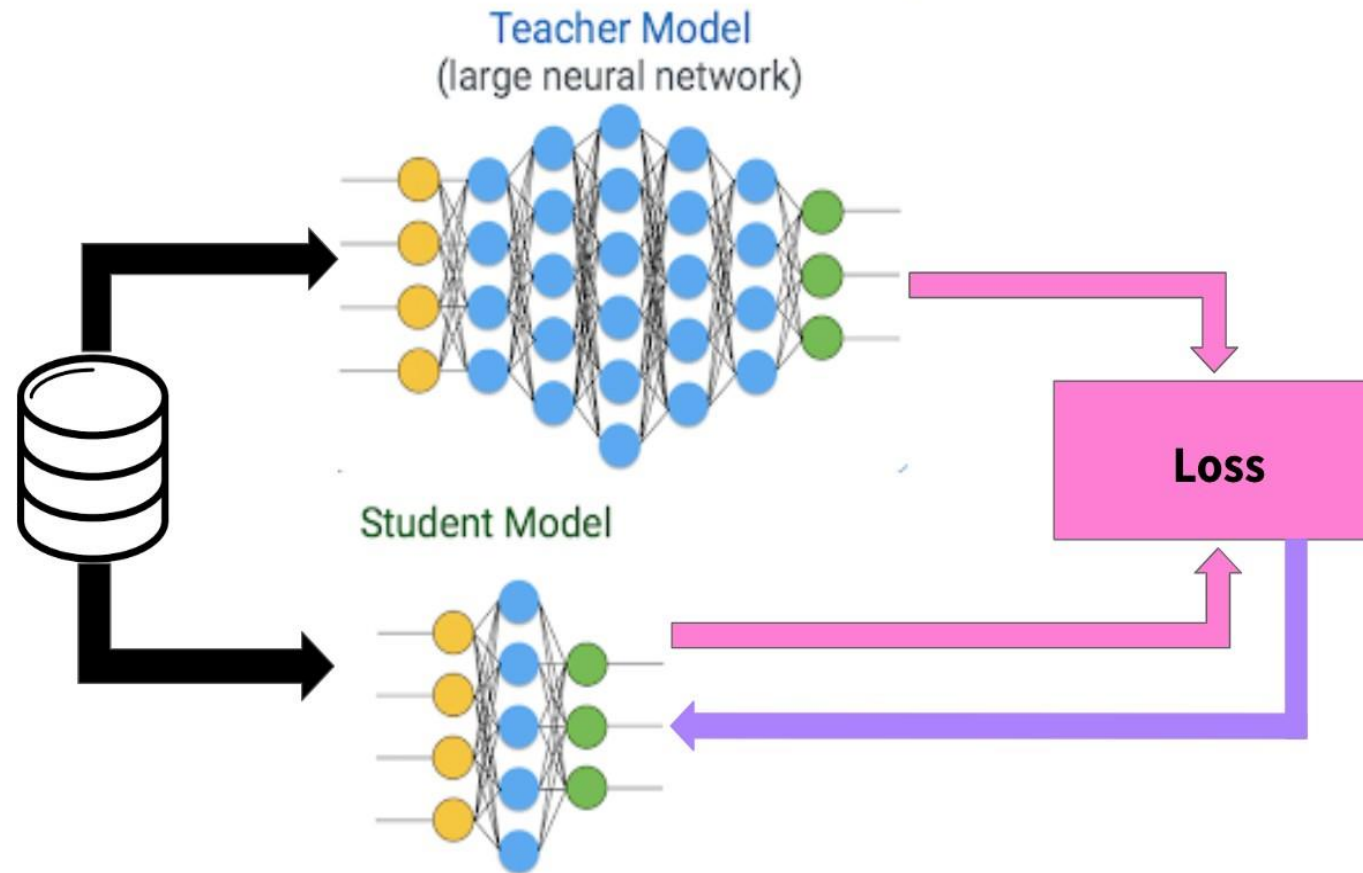
QAT: Quantization Aware Training



Low Rank Approximation

- **Approximate the redundant features of a layer**
 - **By using linear combination of the features**
- **Methods**
 - Singular Value Decomposition
 - Tucker decomposition
 - Canonical Polyadic decomposition

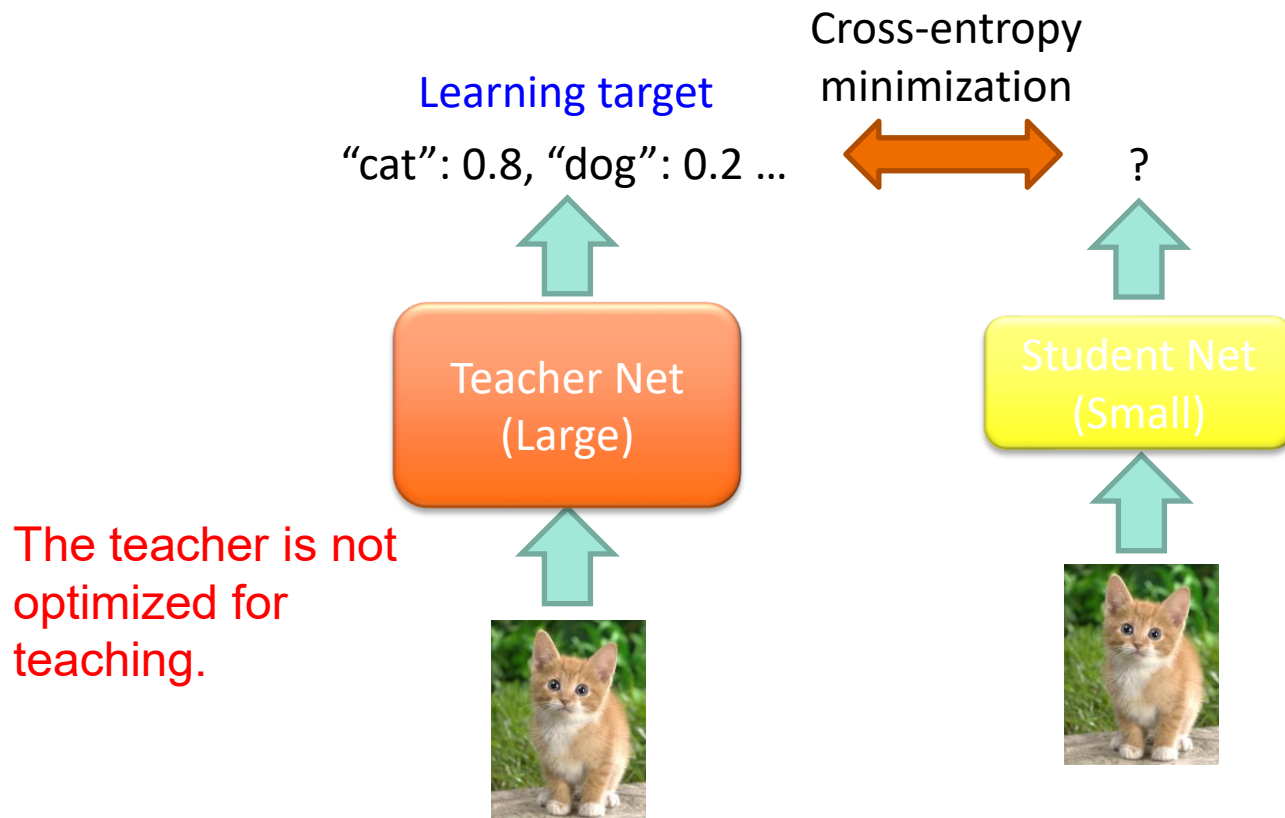
Distillation



Architecture

- most common architectures are:
 - a shallower version of the teacher model with fewer layers and fewer neurons per layer,
 - a quantized version of the teacher model,
 - a smaller network with efficient basic operations,
 - a smaller networks with optimized global network architecture,
 - the same model as the teacher.

Knowledge Distillation



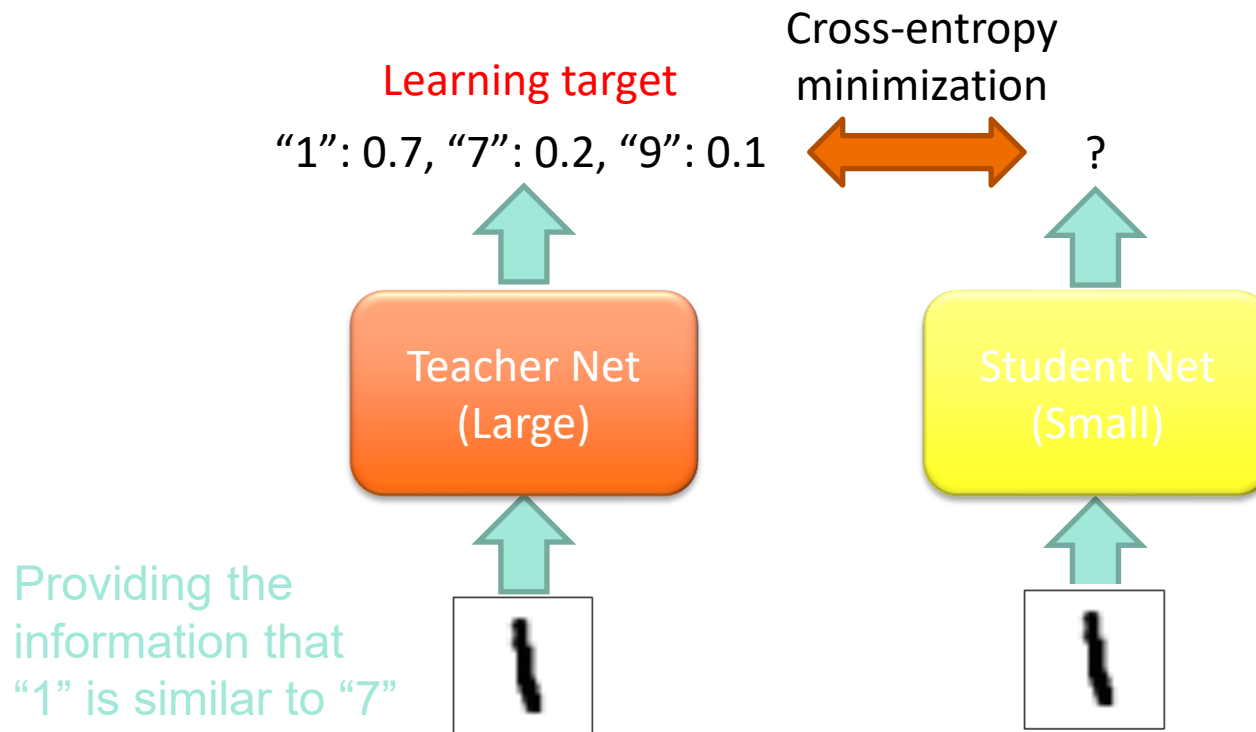
Knowledge Distillation

Knowledge Distillation

<https://arxiv.org/pdf/1503.02531.pdf>

Do Deep Nets Really Need to be Deep?

<https://arxiv.org/pdf/1312.6184.pdf>



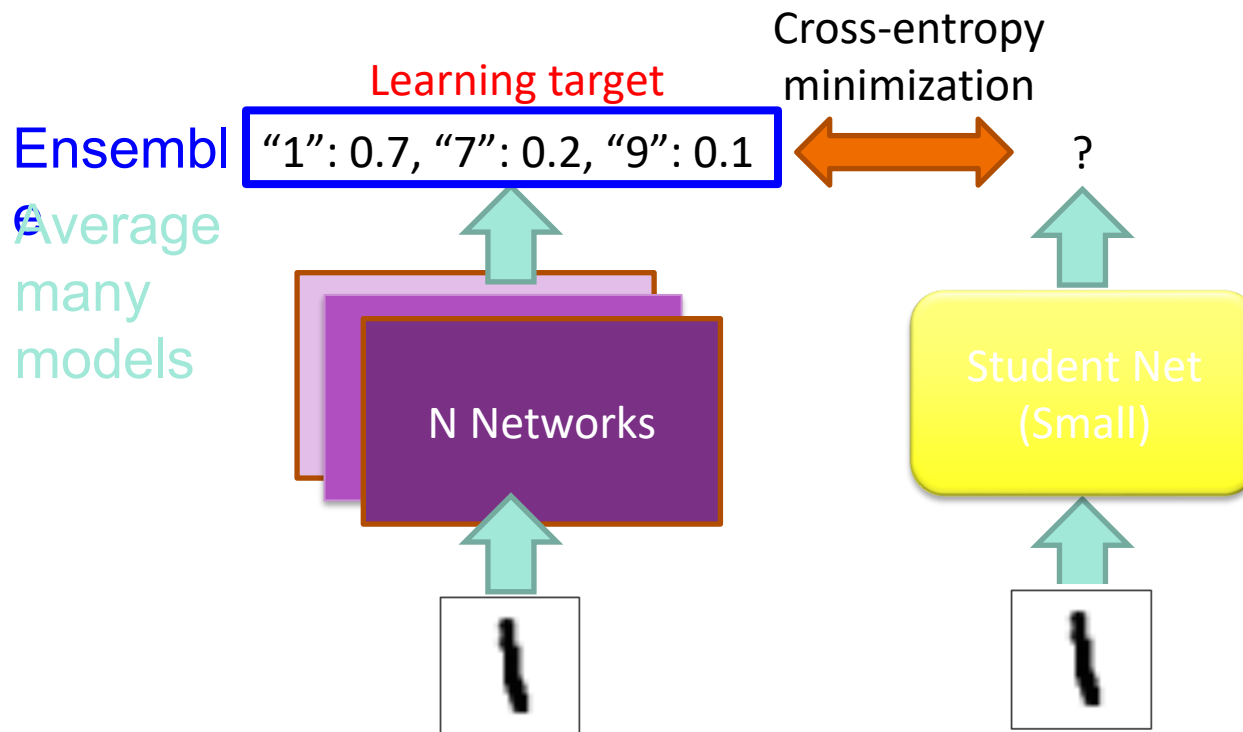
Knowledge Distillation

Knowledge Distillation

<https://arxiv.org/pdf/1503.02531.pdf>

Do Deep Nets Really Need to be Deep?

<https://arxiv.org/pdf/1312.6184.pdf>



Knowledge Distillation

Source of results: <https://arxiv.org/abs/2202.07940>

Table 1. Knowledge distillation is not compatible with strong augmentation. (Das et al., 2020; Cui & Yan, 2021)

Augmentation	Teacher acc.	Student acc.
Normal	73.3	70.4
LabelSmooth (Szegedy et al., 2016)	73.3	68.1
Mixup (Zhang et al., 2017)	74.3	67.9
CutMix (Yun et al., 2019)	76.0	68.8

Table 2. Knowledge distillation suffers from teacher-student gap. (Mirzadeh et al., 2020; Cho & Hariharan, 2019)

Teacher	Teacher acc.	Student (ResNet20) acc.
ResNet32 (He et al., 2016)	71.0	70.8
ResNet56 (He et al., 2016)	73.3	70.4
ResNet110 (He et al., 2016)	75.4	70.5
ResNet164 (He et al., 2016)	75.8	69.9

Can the teacher-network “learn to teach”?

Chain of Thought Reasoning

- Decompose the problem into intermediate steps and solve each before giving the final answer
- *"After Jane gives 2 flowers to her mom she has 10 . . . then after she gives 3 to her dad she will have 7 . . . so the answer is 7."*
- Sufficiently large LMs can generate chains of thought if demonstrations of chain-of-thought reasoning are provided as few-shot prompting.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Fine Tuning

- Supervised Training
 - Transfer Learning

Basic idea of Transfer Learning (cont)

- Train on a big "**source**" data set, with a big model, on one downstream tasks (say classification). Do it once and save the parameters. This is called a **pre-trained model**.
- Use these parameters for other smaller "**target**" datasets, say, for classification on new images (possibly different **domain**, or training distribution), or for image segmentation on old images (new **task**), or new images (new task and new domain).
-

Basic idea of Transfer Learning (cont)

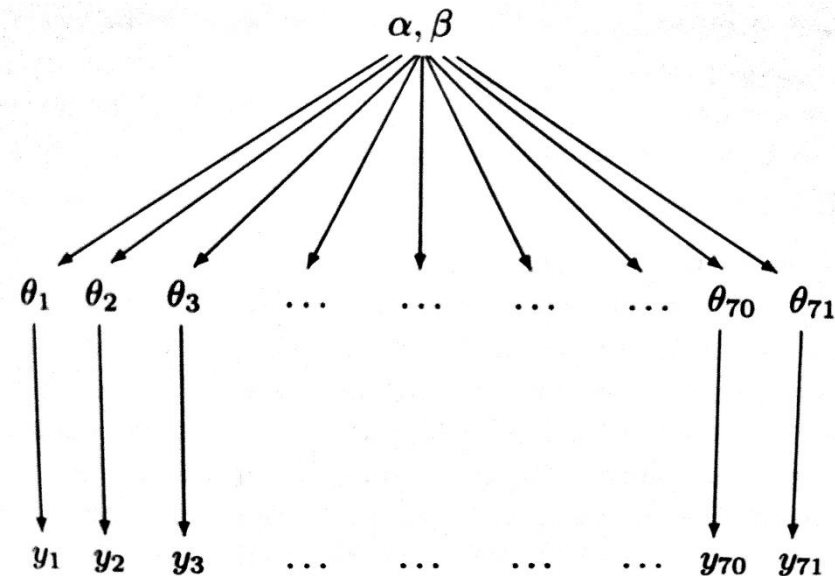
- Train on a big "**source**" data set, with a big model, on one downstream tasks (say classification). Do it once and save the parameters. This is called a **pre-trained model**.
- Use these parameters for other smaller "**target**" datasets, say, for classification on new images (possibly different **domain**, or training distribution), or for image segmentation on old images (new **task**), or new images (new task and new domain).
-
- Less helpful if you have a large target dataset with many labels.
- Will fail if source domain (where you trained big model) has nothing in common with target domain (that you want to train on smaller data set).

Transfer Learning

Not a new idea!

It has been there in the ML and stats literature for a while.

- An exemplar is hierarchical **glm models** in stats, where information flows from higher data units to lower data units to the lower data.
- Neural networks **learn hierarchical representations** and thus are particularly suited to this kind of learning. Furthermore, since we learn representations, we can deal with domain adaptation/covariate shift.



Applications of Transfer Learning

- Learn from simulations (self-driving cars)
- Domain adaptation: bikes -> bikes with backgrounds, bikes at night, etc
- Speech recognition for immigrants and minorities
- Cross-lingual adaptation for few shot learning of resource-poor languages (English->Nepalis, for example)

Applications of Transfer Learning

- Create a classifier to distinguish dogs and cats.
- Use a convnet previously trained (expensive to learn)
 - e.g. Imagenet (1.4 M images and 1000 classes).
- In NLP, you might use a language model trained on Wikipedia and reddit, and then look at legal documents.

Scenarios

Different features spaces among source and target

$$\chi_{source} \neq \chi_{target}$$

Scenario: document A – the source – is written in one language while document B – the target – is written in a different language

Task: can we use the weights learned training a model that distinguish phonemes on the source to distinguish those in the target written in another language? In NLP this method is called cross lingual adaptation.

Scenarios

Different labels among source and target

$$\mathcal{Y}_{source} \neq \mathcal{Y}_{target}$$

Scenario: farm animals versus wild forest animals or different level of classification, e.g. {dogs, cats} different breeds {Retriever, Bulldog, ..., Persian, Siamese}.

Scenarios

Different conditional probabilities distribution among source and target task

$$P_s(Y|X) \neq P_t(Y|X)$$

Scenario: source and target are documents are unbalanced regarding their labels. Common scenario in practice, approachable with sampling techniques (e.g. under, over).

Probability Shift: $P_s(Y) \neq P_t(Y)$ not the same class distribution

Conditional Shift: $P_s(X|Y) \neq P_t(X|Y)$

Key Takeaways

During the process of transfer learning, the following three important questions must be answered:

- **What to transfer?** Identify which portion of knowledge is source-specific and what is common between the source and the target.
- **When to transfer?** Aim at utilizing transfer learning to improve target task performance/results and not degrade them. We need to be careful about when to transfer and when not to.
- **How to transfer?** Changes to existing algorithms and different techniques, which we will cover in later sections of this article.

Transfer Learning Strategies

There are different transfer learning strategies and techniques, which can be applied **based on the domain, task at hand, and the availability of data:**

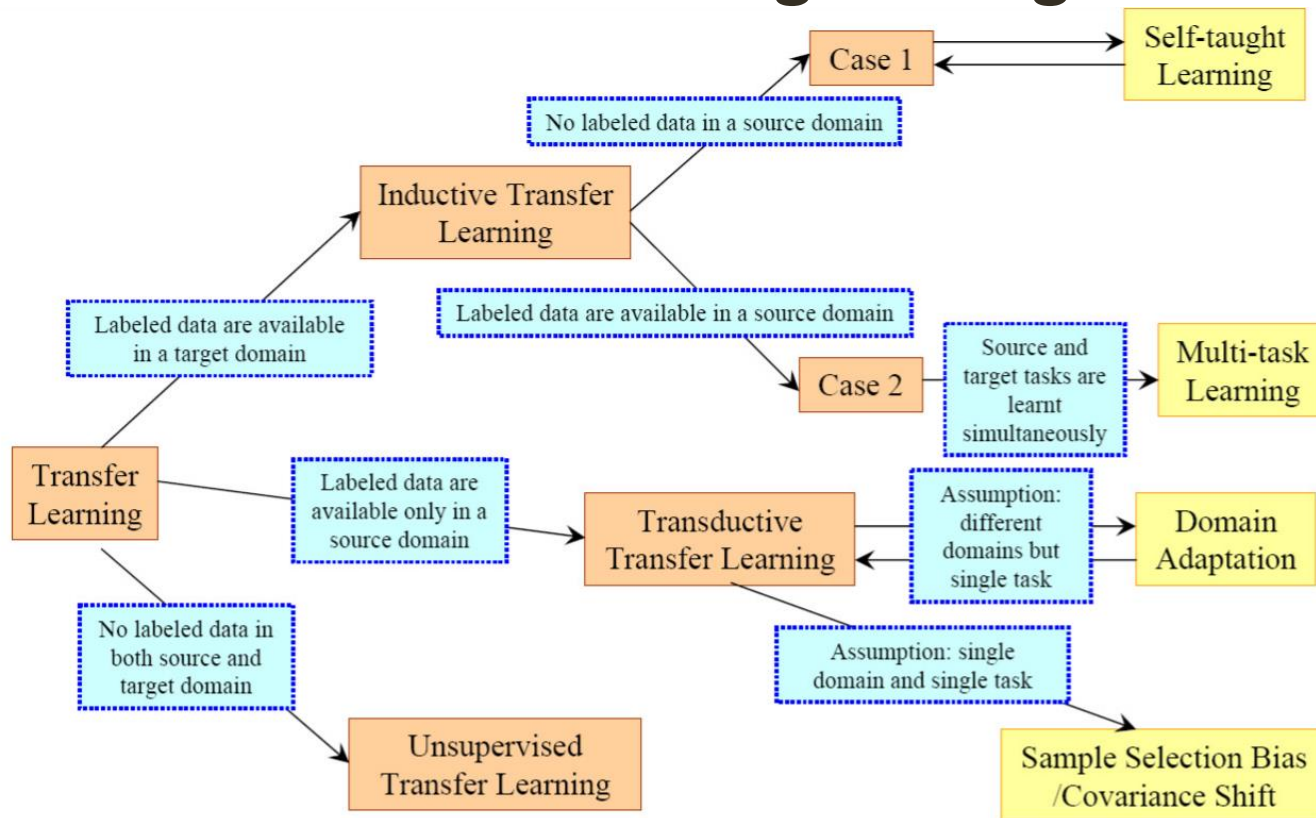
- **Inductive Transfer learning:** the source and target have **same domains**, yet they have **different tasks**. The algorithms utilize the inductive biases of the source domain to help improve the target task.
- **Unsupervised Transfer Learning:** the source and target have **same domains**, with a focus on **unsupervised tasks in the target** domain. The source and target domains are similar, but the tasks are different. In this scenario, labeled data is unavailable in either of the domains.
- **Transductive Transfer Learning:** In this scenario, there are similarities between the source and target tasks, but the corresponding domains are different. In this setting, the source domain has a lot of labeled data, while the target domain has none.

Transfer Learning Strategies

The approaches for Transfer Learning can be defined in few categories:

- **Instance transfer:** Reusing knowledge from the source domain to the target task (ideal scenario). In most cases, the source domain data cannot be reused directly.
- **Feature-representation transfer:** This approach aims to minimize domain divergence and reduce error rates by identifying good feature representations that can be utilized from the source to target domains.
- **Parameter transfer:** This approach works on the assumption that the models for related tasks share some parameters or prior distribution of hyperparameters.
- **Relational-knowledge transfer** attempts to handle non-IID data, such as data that is not independent and identically distributed.

Transfer Learning Strategies



Transfer Learning

What people think

- you can't do deep learning unless you have a million labeled examples.

What people can do, instead

- You can learn representations from unlabeled data
- You can train on a nearby objective for which is easy to generate labels (imageNet).
- You can transfer learned representations from a relate task.

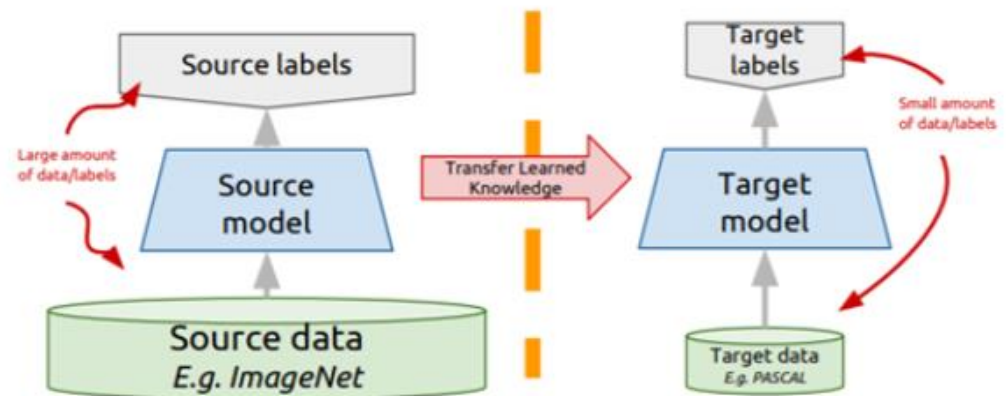
Transfer Learning

Instead of training a network from scratch:

- Take a network trained on a different domain for a different **source task**
- Adapt it for your domain and

Variations

- Same domain, different task.
- Different domain, same task.



Representation Extraction

Use representations learned by big net to extract features from new samples, which are then fed to a new classifier:

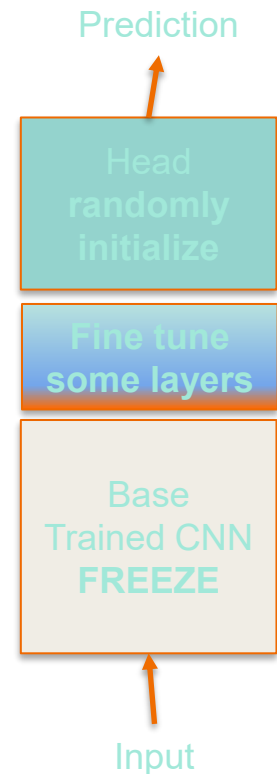
- Keep (frozen) convolutional **base** from big model.
- Generally throw away **head** FC layers since these have no notion of space, and convolutional base is more generic.
- Since there are both dogs and cats in *ImageNet* you could get **away** with using the head FC layers as well. But by throwing it away you can learn more from other dog/cat images.

$$P_s(Y|X) \neq P_t(Y|X)$$



- Fine-tuning
- Up to now we have frozen the entire convolutional base.
 - Remember that earlier layers learn highly generic feature maps (edges, colors, textures).
 - Later layers learn abstract concepts (dog's ear).
 - To particularize the model to our task, its often worth tuning the later layers as well.
 - But we must be very careful not to have big gradient updates.

$$P_S(x) \neq P_t(X)$$



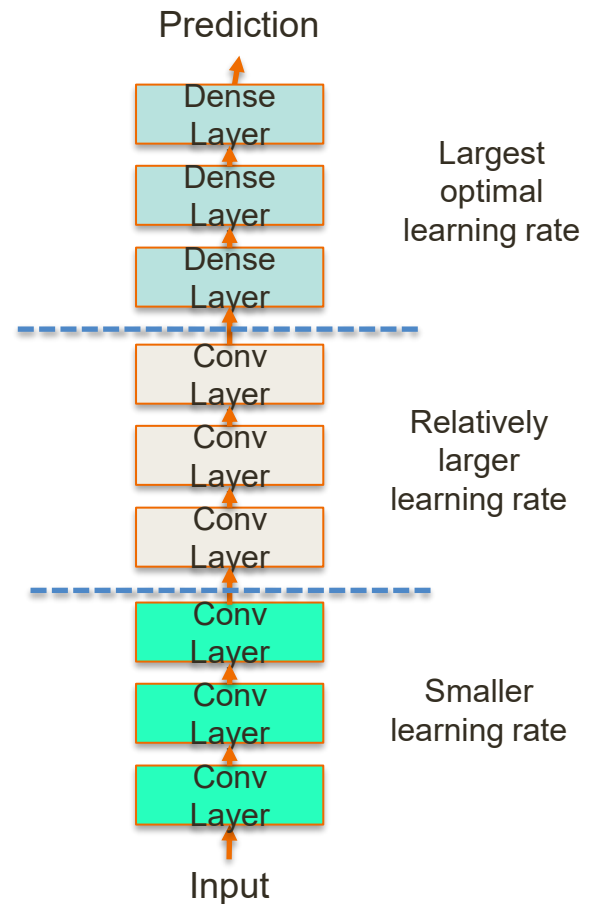
Procedure for Fine-tuning

1. Freeze the model base.
2. First train the fully connected head you added, keeping the convolutional base fixed.
3. Unfreeze some "later" layers in the base net and now train the base net and FC net together.

Since you are now in a better part of the loss surface already, gradients won't be terribly high, but we still need to be careful. Thus often we use a **very low learning rate**.

Transfer Learning for Deep Learning: Differential Learning Rates

- A low learning rate can take a lot of time to train on the "later" layers. Since we trained the FC head earlier, we could probably retrain them at a higher learning rate.
- General Idea: **Train different layers at different rates.**
- Each "earlier" layer or layer group (the color-coded layers in the image) can be trained at 3x-10x smaller learning rate than the next "later" one.
- One could even train the entire network again this way until we overfit and then step back some epochs.



Instruction Fine Tuning

- CoT Prompting
 - What is Prompting?
 - Zero Shot Prompting
 - Chain of Thought Prompting
 - Instruction Prompting